CE 222 Final Project Report

Programming a Finite Element Solver for Shear-Deformable Plates

Kate Kim

May 6, 2026

1 Executive Summary

This report presents the development, verification, and application of a finite element solver for shear-deformable plates. The objective of the project was to compute the transverse deflection at an inner corner of a rectangular plate with a rectangular cutout subjected to a line load along the upper edge of the cutout.

The plate was modeled using a Reissner–Mindlin formulation with three degrees of freedom per node: transverse displacement w, and rotations θ_1 and θ_2. A heterosis-type element was implemented, where the transverse displacement was interpolated with Q8 shape functions and the rotations and geometry were interpolated with Q9 shape functions. The solver was written in Python and includes mesh generation, degree-of-freedom mapping, shape functions and derivatives, Jacobian mapping, Gaussian quadrature, bending and shear stiffness matrices, global assembly, loading, boundary condition enforcement, solution, and post-processing.

The final project model used the specified geometry, material properties, loading, and boundary conditions. Using the refined 36×45 mesh with 1134 elements and 13230 degrees of freedom, the computed transverse deflection at point A was

$$\boxed{w_A = -0.34888 \text{ mm}}$$

and the maximum downward deflection was

$$w_{\min} = -0.52615 \text{ mm}$$

Negative transverse displacement values indicate downward deflection according to the sign convention used in the solver.

The solver was verified using unit tests, patch tests, analytical example problems, and project mesh convergence studies. The unit tests checked individual components of the implementation, including mesh generation, degree-of-freedom mapping, shape function values and derivatives, Jacobian mapping, Gaussian quadrature, B-matrix construction, element stiffness formation, global assembly, load application, and boundary condition enforcement. These tests were used to confirm that the main building blocks of the solver were implemented correctly before solving the full project problem.

Patch tests were then used to verify the element formulation at a more fundamental level. Rigid body translation and rotation tests confirmed that the element produced zero bending and shear strains for zero-strain displacement fields, while the pure bending patch test confirmed that the element could reproduce a constant curvature field with zero shear strain. These tests helped verify that the Q8/Q9 interpolation, coordinate mapping, and strain-displacement matrices were internally consistent.

The analytical and benchmark examples included a simply supported rectangular plate under uniform pressure, a cantilever plate strip subjected to a transverse line load at the free end, and a clamped circular plate subjected to a concentrated center load. These examples checked the solver against known analytical or reference solutions and verified different parts of the finite element implementation. The project mesh convergence study showed that the solution stabilized with mesh refinement, with the percent change in w_A decreasing to 0.370% between the two finest meshes considered. The mesh sizes used in the refinement study were limited by the available computational resources, since finer meshes required larger global stiffness matrices and significantly more memory. Within these practical limits, the results provide confidence that the solver is functioning correctly and that the computed deflection at point A is reasonable.

2 Introduction

The purpose of this project was to develop a finite element solver for shear-deformable plates and apply it to the plate structure shown in Figure 1. The structure consists of a 500 mm $\times$ 300 mm plate with a 250 mm $\times$ 180 mm rectangular cutout. A line load is applied along the upper edge of the cutout, and the quantity of interest is the transverse displacement at point A.

This project was also intended to improve understanding of how finite element solvers work internally. While commercial finite element programs automate mesh generation, element stiffness formation, global assembly, boundary condition enforcement, and solution, implementing these steps directly helps clarify how numerical results are produced and how modeling assumptions affect accuracy.

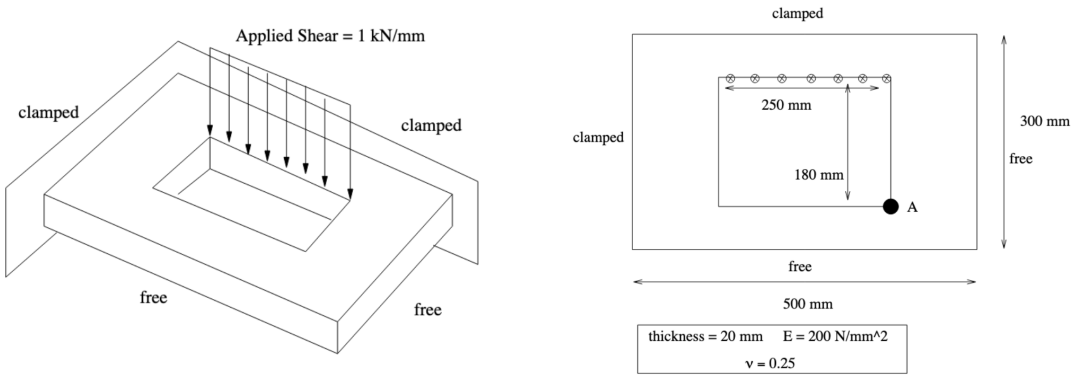

Figure 1: Project plate geometry, loading, boundary conditions, and point A.

3 Modeling Approach and Methods

3.1 Plate Theory

The plate was modeled using Reissner–Mindlin plate theory, which accounts for transverse shear deformation [1, 3]. The primary unknowns are the transverse displacement of the midsurface, $w(x, y)$, and two independent rotations, $\theta_1(x, y)$ and $\theta_2(x, y)$.

The model includes bending curvature terms and transverse shear strain terms. The bending response is governed by the bending constitutive matrix, while the shear response is governed by the shear constitutive matrix with a shear correction factor. The governing strain-displacement relations, weak form, and finite element equations are provided in Appendix A.

3.2 Finite Element Discretization

A heterosis-type element was used. The transverse displacement w was interpolated using Q8 serendipity shape functions, while the rotations θ_1 and θ_2, as well as the geometry, were interpolated using Q9 shape functions.

This gives a 26 degree-of-freedom element:

$$8\ w\text{-DOFs} + 9\ \theta_1\text{-DOFs} + 9\ \theta_2\text{-DOFs} = 26\ \text{DOFs}$$

The bending stiffness was evaluated using 3×3 Gaussian quadrature, while the shear stiffness was evaluated using 2×2 Gaussian quadrature. Additional details on the Q8/Q9 interpolation, B-matrices, and element stiffness formulation are provided in Appendix A. The finite element implementation follows the standard displacement-based finite element procedure [1, 2, 3].

3.3 Mesh Generation

A structured Q9 mesh was generated over the rectangular plate domain. The rectangular cutout was handled by removing elements within the cutout region and deactivating nodes strictly inside the opening. Nodes along the cutout boundary were retained so that the geometry and line load could be represented correctly.

The final project mesh is shown in Figure 2. The mesh was aligned with the cutout boundaries to avoid partially cut elements and to ensure that the applied line load was assembled along the correct edge.

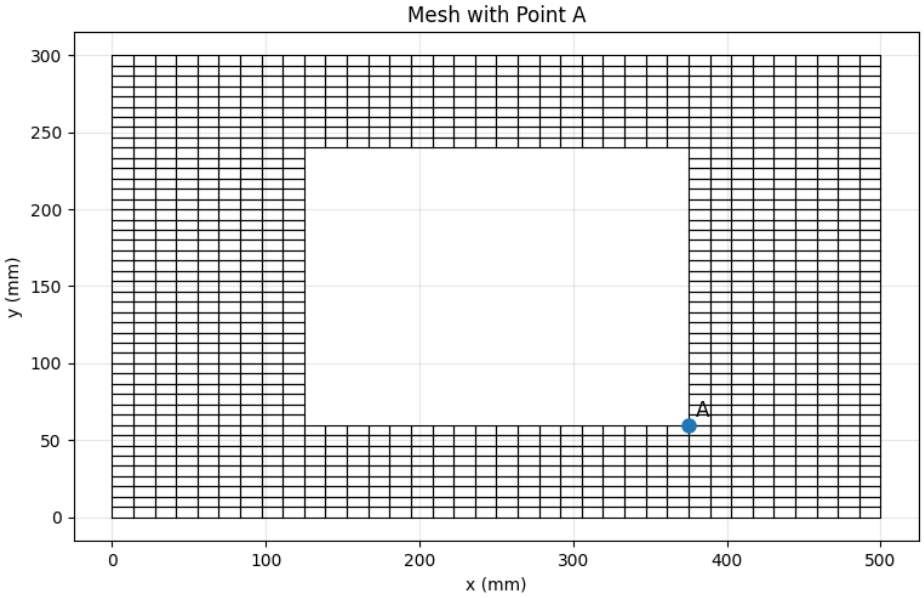

Figure 2: Final finite element mesh used for the project problem. The mesh is aligned with the rectangular cutout boundaries so that the opening geometry and cutout-edge line load are represented correctly.

3.4 Boundary Conditions and Loading

The top and left edges of the plate were modeled as clamped boundaries. Therefore, w, θ_1, and θ_2 were prescribed as zero along those boundaries. The right and bottom edges were modeled as free boundaries.

The applied line load was placed along the upper edge of the rectangular cutout and assembled only into the transverse displacement DOFs. The total applied load was checked during the analysis and matched the expected value of -250000 N. The detailed consistent line-load expression is provided in Appendix G.

A summary of the main solver functions, including their inputs, outputs, roles, and verification methods, is provided in Appendix H.

4 Results and Accuracy Assessment

4.1 Final Project Results

The final model was run using the project input parameters summarized in Table 1. Negative transverse displacement values indicate downward deflection according to the sign convention used in the solver.

Table 1: Final project input parameters.

Parameter	Value
Plate width, L_x	500 mm
Plate height, L_y	300 mm
Cutout width	250 mm
Cutout height	180 mm
Elastic modulus, E	200000 N/mm^2
Poisson's ratio, ν	0.25
Thickness, t	20 mm
Line load, $\bar{q}$	-1000 N/mm
Total applied load	-250000 N
Clamped boundaries	Left and top edges
Free boundaries	Right and bottom edges

Table 2 summarizes the key output quantities for the final mesh.

Table 2: Final analysis results for project problem.

Quantity	Value
Number of elements	1134
Number of DOFs	13230
Maximum downward deflection	-0.52615 mm
Transverse deflection at point A	-0.34888 mm
Total applied load	-250000 N

The computed transverse displacement at point A was

$$\boxed{w_A = -0.34888 \text{ mm}}$$

Figure 3 shows the transverse displacement field for the final mesh. The largest downward deflections occur near the loaded cutout edge, which is consistent with the applied loading and support conditions.

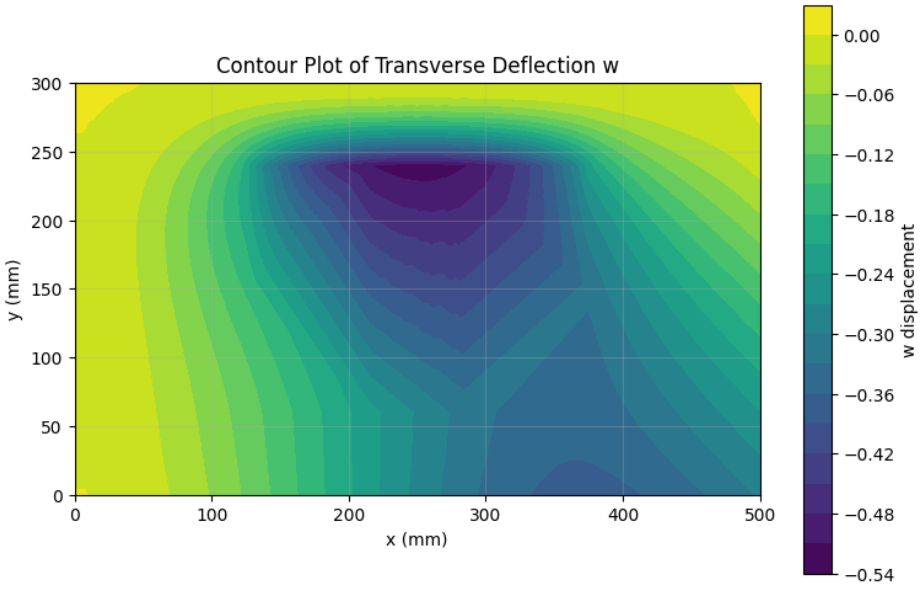

Figure 3: Contour plot of transverse displacement w for the final project mesh. The largest downward deflections occur near the loaded cutout edge, which is physically consistent with the applied line load and boundary conditions.

4.2 Unit Tests

Before using the solver for the final project problem, unit tests were performed to verify that individual components of the code were working correctly. These tests were organized around the main parts of the finite element workflow.

The mesh generation tests checked the structured Q9 mesh, cutout handling, element connectivity, and boundary tagging. These tests verified `generate_mesh()`, `generate_nodes()`, `get_q9_connectivity()`, and the cutout-checking routines.

The degree-of-freedom mapping tests checked that active nodes were assigned the correct DOFs and that inactive nodes were excluded. They also verified that the Q9 center node correctly had rotational DOFs but no active transverse displacement DOF. These tests verified `assign_dofs()` and `get_element_dofs()`.

The shape function tests checked partition of unity and the Kronecker delta property for both Q8 and Q9 shape functions. The derivative tests checked that derivative sums were zero and compared analytical derivatives against numerical finite-difference derivatives. These tests verified `shape_Q8()`, `shape_Q9()`, `shape_Q8_derivatives()`, and `shape_Q9_derivatives()`.

The Jacobian and coordinate mapping tests checked that derivatives were correctly transformed from the parent element to the physical element and that the Jacobian determinant remained positive. These tests verified `jacobian()`, `jacobian_inverse()`, `map_derivatives()`, and `jacobians()`.

The Gaussian quadrature tests checked that the implemented quadrature rules integrated simple known functions correctly. These tests verified `gauss_1D()` and `gauss_2D()`.

The B-matrix and material matrix tests checked that the bending and shear B-matrices had the correct dimensions and that the material matrices had the correct sizes and symmetry. These tests

verified `build_B_bending()`, `build_B_shear()`, and `build_D_matrix()`.

The element stiffness tests checked that the element stiffness matrix had the correct size, was symmetric, contained finite values, and had no all-zero rows. These tests verified `element_stiffness()`.

Finally, the global assembly, load, and boundary condition tests checked that the global stiffness matrix was assembled with the correct size and symmetry, that the total applied line load was correct, and that prescribed boundary conditions were enforced properly. These tests verified `assemble()`, `assemble_system()`, `edge_load_vector()`, `apply_cutout_top_line_load()`, `fixed_dofs()`, and `bc()`.

Overall, the unit tests helped confirm that each individual component of the solver behaved as expected before the full project problem was solved. The detailed criteria used in the unit tests are included in Appendix B, and the corresponding solver functions are summarized in Appendix H.

4.3 Patch Tests

Patch tests were used to verify the element-level formulation by checking whether simple prescribed displacement fields produced the expected bending and shear strain states. These tests are important because they verify the consistency of the finite element formulation. In particular, a correct element should represent rigid body motions without producing strain and should reproduce constant strain fields exactly.

Three patch tests were performed: rigid body translation, rigid body rotation, and pure bending. The rigid body translation test confirmed that a constant transverse displacement produced zero bending curvature and zero transverse shear strain. The rigid body rotation test confirmed that a linear transverse displacement field with compatible constant rotations also produced zero bending and shear strain, up to numerical roundoff. The pure bending test confirmed that a quadratic transverse displacement field with compatible rotations produced the expected constant curvature field and zero shear strain.

These patch tests verified the most important element-level functions in the program: `shape_Q8()`, `shape_Q9()`, `shape_Q8_derivatives()`, `shape_Q9_derivatives()`, `jacobians()`, `map_derivatives()`, `build_B_bending()`, `build_B_shear()`, `assign_dofs()`, and `get_element_dofs()`. These functions determine how nodal displacement fields are interpolated, mapped to physical coordinates, and converted into bending curvatures and shear strains.

The patch test outputs showed that the prescribed displacement fields produced the expected strain states. This provides evidence that the element formulation is internally consistent and that the solver is correctly converting nodal DOFs into strain fields. Detailed patch test equations and output values are included in Appendix C, and the functions exercised by the patch tests are summarized in Appendix H.

4.4 Analytical Example and Benchmark Problems

In addition to the unit tests and patch tests, three example problems were solved to verify the full finite element workflow against analytical or reference solutions. These examples were selected because they test different parts of the solver while still using the same core finite element routines used in the final project problem, including shape functions, derivative mapping, Jacobian calculation, B-matrix construction, element stiffness formation, global assembly, boundary condition

enforcement, loading, solution, and post-processing.

4.4.1 Simply Supported Rectangular Plate Under Uniform Pressure

The first example was a simply supported square plate subjected to a uniform transverse pressure. This problem has a classical Navier thin-plate solution for the center deflection [3]. In the finite element model, the simply supported condition was imposed by fixing only the transverse displacement w along all four edges while leaving the rotations free. This example verified that the solver can apply distributed pressure loading, enforce pinned boundary conditions, and reproduce a known plate bending solution. The analytical solution, numerical results, convergence table, and convergence plots for this example are provided in Appendix D.

4.4.2 Cantilever Plate Strip Under End Line Load

The second example was a cantilever plate strip with a transverse line load applied along the free end. The plate was clamped along one short edge and left free along the other three edges. The numerical tip displacement was compared against Euler–Bernoulli and Timoshenko beam-strip theory [3]. This example verified the solver's treatment of clamped boundary conditions, free boundaries, edge line loading, bending deformation, and shear deformation. Since the model is a plate strip rather than a full two-dimensional closed-form plate solution, this example is best interpreted as an engineering benchmark rather than an exact plate solution. The beam-strip reference solution, numerical results, convergence table, and convergence plots are provided in Appendix E.

4.4.3 Clamped Circular Plate With Center Load

The third example was a clamped circular plate subjected to a concentrated center load. This benchmark has an analytical thin-plate solution for the center displacement [3]. Only one quadrant of the circular plate was modeled using symmetry, with the circular edge clamped and symmetry conditions applied along the two radial edges. This example verified the full solver workflow on a non-rectangular, curved geometry. Although the circular plate required a different mesh generator and boundary condition selector, it used the same core element routines as the project problem, including the Q8/Q9 interpolation, Jacobian mapping, B-matrix construction, element stiffness calculation, global assembly, and solution procedure. The analytical solution, numerical results, convergence table, and plots are provided in Appendix F.

4.5 Project Mesh Convergence Study

A mesh refinement study was performed by solving the project problem with increasing mesh density. The primary quantities tracked were the maximum downward deflection and the transverse deflection at point A.

Table 3: Mesh convergence study results for the project problem.

n_x	n_y	Elements	DOFs	$w_{\min}$ (mm)	w_A (mm)
4	5	14	238	-0.558031	-0.382826
12	15	126	1638	-0.525851	-0.355190
24	30	504	6048	-0.526068	-0.350175
36	45	1134	13230	-0.526154	-0.348881

Table 4: Percent change between successive project mesh refinements.

Mesh refinement	$w_{\min}$ change (%)	w_A change (%)
$4 \times 5 \rightarrow 12 \times 15$	5.767	7.219
$12 \times 15 \rightarrow 24 \times 30$	0.041	1.412
$24 \times 30 \rightarrow 36 \times 45$	0.016	0.370

Figure 4 shows the convergence of the displacement response with mesh refinement. The results show that both $w_{\min}$ and w_A stabilize as the mesh is refined. The change in w_A decreases to 0.370% between the two finest meshes, indicating that the final mesh provides a sufficiently converged estimate of the displacement at point A.

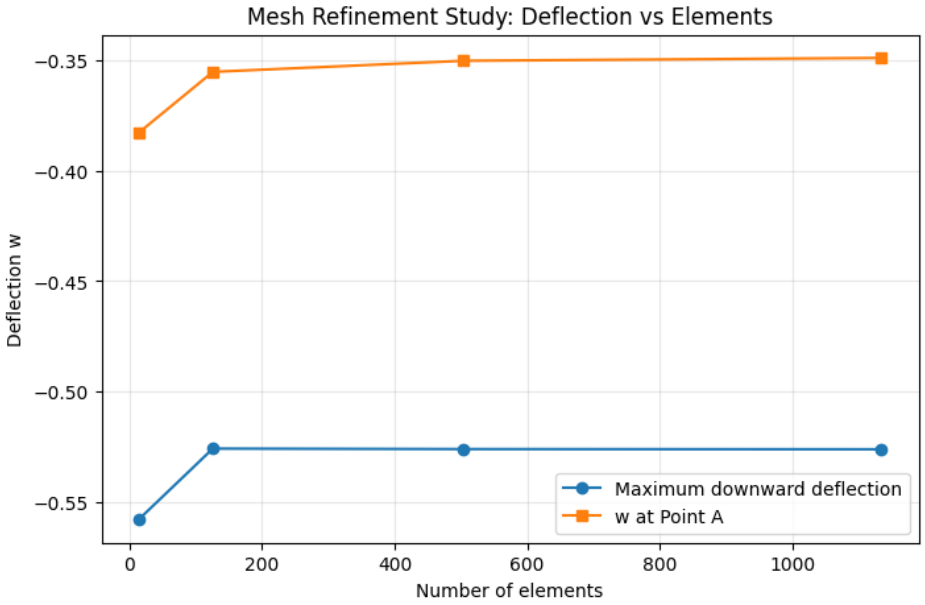

Figure 4: Mesh convergence of maximum downward deflection and deflection at point A. Both quantities stabilize as the mesh is refined, supporting the use of the 36×45 mesh for the final result.

The 36×45 mesh was selected as the final mesh because the change in w_A from the previous refinement level was only 0.370%, indicating that the displacement at point A had effectively stabilized. The maximum downward displacement $w_{\min}$ also changed by only 0.016% between the

two finest meshes. This suggests that the reported final deflection is sufficiently converged for the purpose of this project.

4.6 Accuracy Assessment

The accuracy of the final result was assessed using multiple levels of verification. First, unit tests were used to check individual code components such as mesh generation, DOF mapping, shape functions, derivatives, Jacobians, quadrature, B-matrices, stiffness matrices, load assembly, and boundary condition enforcement. Second, patch tests were used to confirm that the element formulation correctly reproduces rigid body motion and constant curvature fields. Third, the three analytical example and benchmark problems were used to compare the full solver workflow against analytical or reference solutions. Finally, the project mesh convergence study confirmed that the deflection at point A stabilizes with mesh refinement.

The simply supported rectangular plate verified distributed pressure loading and pinned boundary conditions. The cantilever plate strip verified clamped boundary conditions, free edges, edge line loading, and shear-deformable behavior. The clamped circular plate verified the solver on a curved geometry and checked convergence toward a known analytical plate solution. Together, these tests provide evidence that the solver is functioning correctly and that the final computed displacement is a reasonable finite element result.

Some limitations remain. The mesh is structured and must be aligned with the cutout boundaries to apply the line load correctly. Very fine meshes also become memory-intensive because the global stiffness matrix is currently assembled as a dense matrix. A sparse matrix implementation would allow larger refinement studies in future work.

5 Conclusion

A finite element solver for shear-deformable Reissner–Mindlin plates was developed and applied to a plate with a rectangular cutout. The solver uses a heterosis-type Q8/Q9 element, where transverse displacement is interpolated with Q8 shape functions and rotations are interpolated with Q9 shape functions.

The final computed transverse displacement at point A was

$$\boxed{w_A = -0.34888 \text{ mm}}$$

and the maximum downward displacement was

$$w_{\min} = -0.52615 \text{ mm}$$

The solution was supported by unit tests, patch tests, analytical example problems, and mesh convergence studies. The benchmark and convergence results indicate that the solver is producing stable and physically reasonable results.

Through this project, I gained a clearer understanding of how finite element plate solvers construct shape functions, map derivatives, form element stiffness matrices, assemble global systems, impose boundary conditions, apply consistent loads, and assess numerical accuracy. The project

also showed the importance of verifying individual solver components before relying on full-model results.

6 References

References

[1] Zienkiewicz, O. C., Taylor, R. L., and Govindjee, S. *The Finite Element Method*. Elsevier.

[2] Hughes, T. J. R. *The Finite Element Method: Linear Static and Dynamic Finite Element Analysis*. Dover Publications.

[3] CE 222 Course Lecture Notes, University of California, Berkeley, Spring 2026.

A Appendix: Governing Equations and Element Formulation Details

The Reissner–Mindlin plate formulation uses the displacement vector

$$\boldsymbol{u} = \begin{bmatrix} w & \theta_1 & \theta_2 \end{bmatrix}^T$$

where w is the transverse displacement and θ_1, θ_2 are rotations of the plate normal.

The bending curvatures are

$$\boldsymbol{\kappa} = \begin{bmatrix} \dfrac{\partial \theta_1}{\partial x} \\ \dfrac{\partial \theta_2}{\partial y} \\ \dfrac{\partial \theta_1}{\partial y} + \dfrac{\partial \theta_2}{\partial x} \end{bmatrix}$$

and the transverse shear strains are

$$\boldsymbol{\gamma} = \begin{bmatrix} \dfrac{\partial w}{\partial x} - \theta_1 \\ \dfrac{\partial w}{\partial y} - \theta_2 \end{bmatrix}$$

The bending constitutive matrix is

$$\boldsymbol{D}_b = \frac{Et^3}{12(1-\nu^2)} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \dfrac{1-\nu}{2} \end{bmatrix}$$

and the shear constitutive matrix is

$$\boldsymbol{D}_s = \kappa G t \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

where $G = E/[2(1+\nu)]$ and $\kappa = 5/6$.

The weak form is

$$\int_\Omega \delta\boldsymbol{\kappa}^T \boldsymbol{D}_b \boldsymbol{\kappa}\, d\Omega + \int_\Omega \delta\boldsymbol{\gamma}^T \boldsymbol{D}_s \boldsymbol{\gamma}\, d\Omega = \int_\Omega \delta w q\, d\Omega + \int_\Gamma \delta w \bar{q}\, ds$$

The transverse displacement and rotations are interpolated as

$$w(x,y) = \sum_{i=1}^{8} N_i^{Q8} w_i$$

$$\theta_1(x,y) = \sum_{i=1}^{9} N_i^{Q9} \theta_{1i}, \qquad \theta_2(x,y) = \sum_{i=1}^{9} N_i^{Q9} \theta_{2i}$$

The bending and shear strains are written in matrix form as

$$\boldsymbol{\kappa} = \boldsymbol{B}_b \boldsymbol{u}_e, \qquad \boldsymbol{\gamma} = \boldsymbol{B}_s \boldsymbol{u}_e$$

The element stiffness matrix is

$$\boldsymbol{K}_e = \int_{\Omega_e} \boldsymbol{B}_b^T \boldsymbol{D}_b \boldsymbol{B}_b\, d\Omega + \int_{\Omega_e} \boldsymbol{B}_s^T \boldsymbol{D}_s \boldsymbol{B}_s\, d\Omega$$

After assembly, the global finite element system is

$$\boldsymbol{K}\boldsymbol{u} = \boldsymbol{F}$$

B Appendix: Unit Test Details

This appendix provides the technical criteria used in the unit tests. The functions associated with each group of unit tests are summarized in Appendix H.

B.1 Mesh Generation Tests

The mesh generation tests checked that the structured grid dimensions followed

$$N_x = 2n_x + 1, \qquad N_y = 2n_y + 1$$

and that each Q9 element contained 9 nodes. For cutout meshes, the tests checked that no active node was located strictly inside the cutout and that no retained element centroid was inside the cutout.

B.2 Degree-of-Freedom Mapping Tests

The DOF mapping tests checked that global DOF numbers were unique and continuous from 0 to `ndof` -1. Each element was also checked to have 26 active local DOFs:

$$8\ w\text{-DOFs} + 9\ \theta_1\text{-DOFs} + 9\ \theta_2\text{-DOFs} = 26\ \text{DOFs}$$

B.3 Shape Function Tests

The Q8 and Q9 shape functions were checked for partition of unity:

$$\sum_i N_i = 1$$

The Kronecker delta property was also checked at the element nodes.

B.4 Derivative Tests

The derivative tests checked that the derivative of a constant field was zero:

$$\sum_i \frac{\partial N_i}{\partial \xi} = 0, \qquad \sum_i \frac{\partial N_i}{\partial \eta} = 0$$

The analytical derivatives were also compared to numerical finite-difference derivatives.

B.5 Jacobian Tests

For a rectangular element, the expected mapping gives

$$\frac{\partial x}{\partial \xi} = \frac{L_x}{2}, \qquad \frac{\partial y}{\partial \eta} = \frac{L_y}{2}$$

and

$$\det J = \frac{L_x L_y}{4}$$

The Jacobian determinant was checked to be positive, and mapped physical derivatives were checked to sum to zero.

B.6 Quadrature Tests

The 1D Gauss weights were checked to sum to 2, and the 2D Gauss weights were checked to sum to 4. The quadrature rules were also tested by integrating simple polynomial functions with known exact values.

B.7 B-Matrix and Material Matrix Tests

For the 26-DOF element, the bending B-matrix was checked to have size

$$3 \times 26$$

and the shear B-matrix was checked to have size

$$2 \times 26$$

The bending material matrix was checked to have size 3×3, and the shear material matrix was checked to have size 2×2. Both were checked for symmetry.

B.8 Element Stiffness Tests

The element stiffness matrix was checked to have size 26×26, to be symmetric, to contain finite values, and to have no all-zero rows.

B.9 Global Assembly, Load, and Boundary Condition Tests

The global stiffness matrix was checked to have size `ndof` $\times$ `ndof`, to be symmetric, and to contain no NaN or infinite values. The applied cutout line load was checked against

$$F_{\text{total}} = (-1000 \text{ N/mm})(250 \text{ mm}) = -250000 \text{ N}$$

Boundary condition enforcement was checked by confirming that constrained DOFs were properly modified in the global stiffness matrix and global force vector.

C Appendix: Patch Test Details

This appendix provides the detailed displacement fields and expected outputs for the patch tests. The patch tests exercised the interpolation, derivative mapping, and B-matrix routines summarized in Appendix H.

C.1 Rigid Body Translation

The rigid translation patch test used

$$w = c, \qquad \theta_1 = 0, \qquad \theta_2 = 0$$

The expected result is

$$\boldsymbol{\kappa} = \mathbf{0}, \qquad \boldsymbol{\gamma} = \mathbf{0}$$

The computed result was

$$\boldsymbol{\kappa} = \begin{bmatrix} 0 & 0 & 0 \end{bmatrix}^T, \qquad \boldsymbol{\gamma} = \begin{bmatrix} 0 & 0 \end{bmatrix}^T$$

C.2 Rigid Body Rotation

The rigid rotation patch test used

$$w = ax + by, \qquad \theta_1 = a, \qquad \theta_2 = b$$

The expected result is again

$$\boldsymbol{\kappa} = \mathbf{0}, \qquad \boldsymbol{\gamma} = \mathbf{0}$$

The computed shear strains were on the order of 10^{-17}, which was treated as numerical roundoff.

C.3 Pure Bending Patch Test

The pure bending patch test used

$$w = \frac{1}{2}k_x x^2 + \frac{1}{2}k_y y^2 + k_{xy} xy$$

with compatible rotations

$$\theta_1 = \frac{\partial w}{\partial x} = k_x x + k_{xy} y$$

and

$$\theta_2 = \frac{\partial w}{\partial y} = k_y y + k_{xy} x$$

The expected curvature field is

$$\boldsymbol{\kappa} = \begin{bmatrix} k_x \\ k_y \\ 2k_{xy} \end{bmatrix}$$

and the expected shear strain is

$$\boldsymbol{\gamma} = \mathbf{0}$$

For the values used in the test, the expected and computed curvatures were

$$\boldsymbol{\kappa} = \begin{bmatrix} 0.001 \\ -0.0005 \\ 0.0004 \end{bmatrix}$$

The computed shear strains were on the order of 10^{-16}, which was treated as numerical roundoff.

Table 5: Patch test summary.

Patch Test	Expected Result	Status
Rigid body translation	Zero bending and shear strain	Pass
Rigid body rotation	Zero bending and shear strain	Pass
Pure bending	Constant curvature and zero shear strain	Pass

D Appendix: Analytical Example Problem – Simply Supported Rectangular Plate

A simply supported rectangular plate under uniform pressure was used as an analytical verification problem. The model was a square plate with side lengths

$$a = b = 100$$

with material properties

$$E = 200000, \qquad \nu = 0.25, \qquad t = 1.0$$

and uniform pressure

$$q = -1.0$$

The simply supported boundary condition was imposed by fixing only the transverse displacement w on all four edges. The rotations were left free.

For a simply supported rectangular plate under uniform pressure, the analytical thin-plate Navier solution is

$$w(x,y) = \sum_{m=1,3,5,\ldots} \sum_{n=1,3,5,\ldots} \frac{16q}{\pi^6 Dmn \left[\left(\frac{m}{a}\right)^2 + \left(\frac{n}{b}\right)^2 \right]^2} \sin\left(\frac{m\pi x}{a}\right) \sin\left(\frac{n\pi y}{b}\right)$$

where

$$D = \frac{Et^3}{12(1-\nu^2)}$$

is the plate flexural rigidity.

The center displacement was evaluated at

$$x = \frac{a}{2}, \qquad y = \frac{b}{2}$$

The 12×12 mesh generated can be seen in Figure 5, for which the numerical center displacement was

$$w_{\text{center,num}} = -23.0730$$

and the analytical center displacement was

$$w_{\text{center,exact}} = -22.8507$$

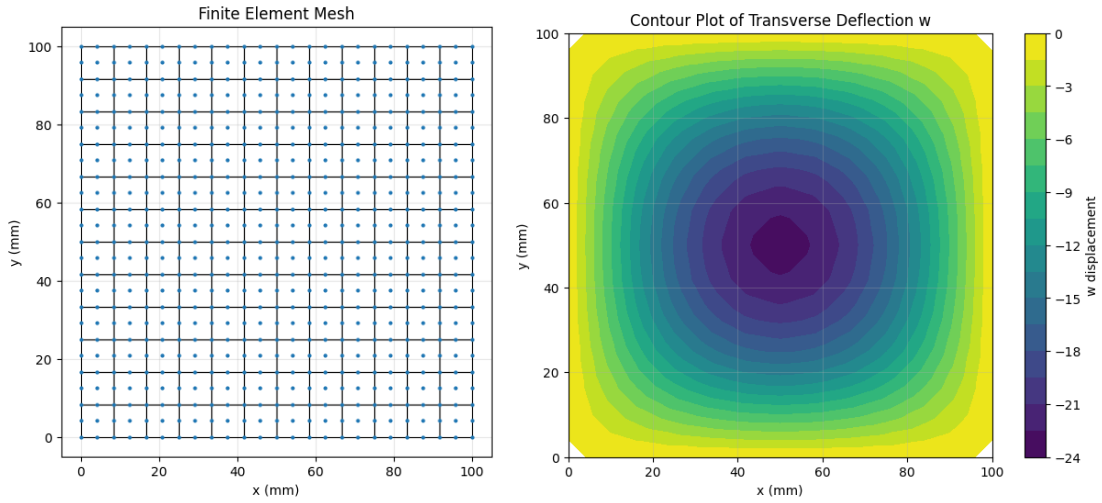

Figure 5: Finite element mesh and contour plot of transverse displacement for a 12×12 simply supported and uniformly loaded plate.

The percent error was

$$0.9728\%$$

The total applied load was also checked:

$$F_{\text{total}} = qab = (-1.0)(100)(100) = -10000$$

which matched the total assembled finite element load.

Table 6: Simply supported uniform load convergence results.

n_x	n_y	Elements	DOFs	$w_{\text{center,num}}$	Error (%)
4	4	16	227	-23.53977	3.0154
8	8	64	803	-23.11047	1.1367
12	12	144	1731	-23.07303	0.9728
16	16	256	3011	-23.07152	0.9662
20	20	400	4643	-23.07162	0.9667

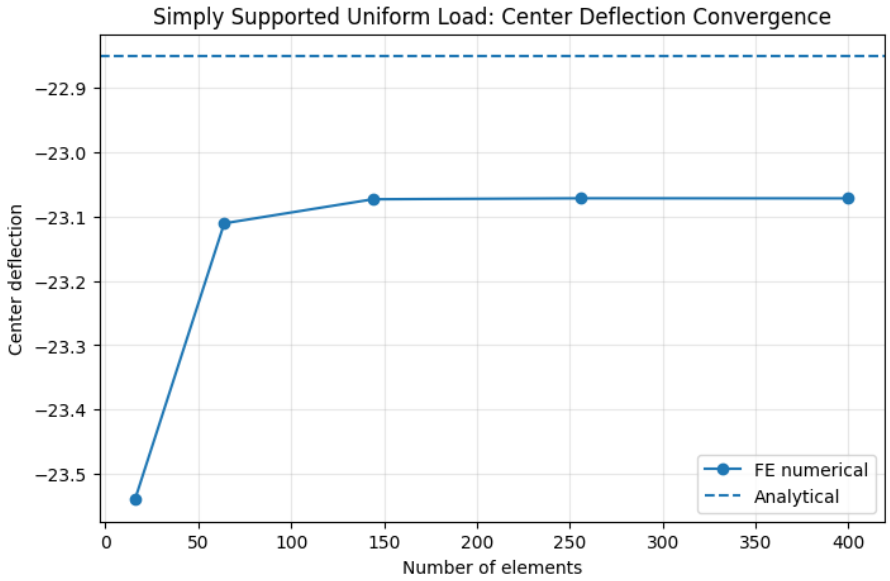

Figure 6: Convergence of center deflection for the simply supported rectangular plate under uniform pressure. The finite element solution approaches the analytical Navier solution as the mesh is refined.

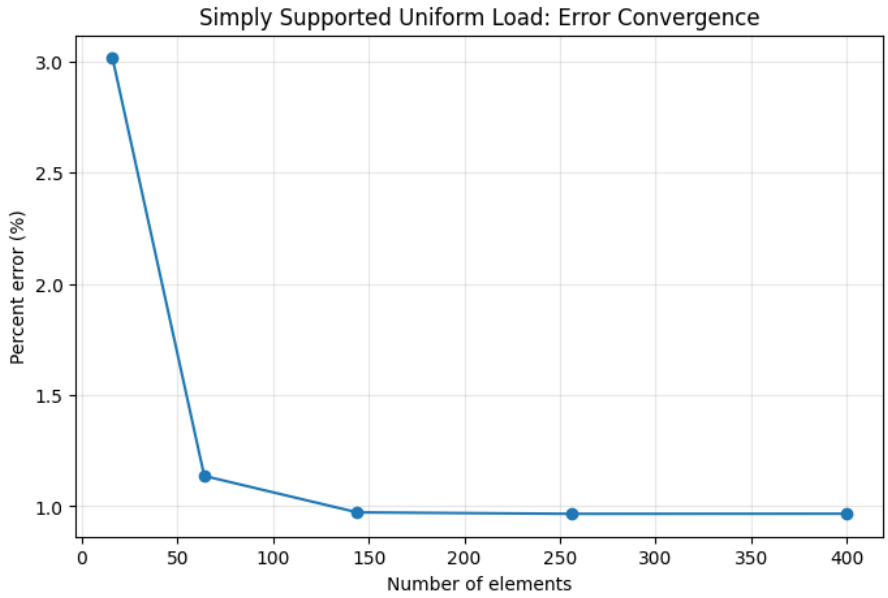

Figure 7: Percent error convergence for the simply supported rectangular plate under uniform pressure. The error decreases and stabilizes below approximately 1%, indicating agreement with the analytical thin-plate solution.

E Appendix: Analytical Example Problem – Cantilever Plate Strip

A cantilever plate strip with a line load at the free end was used as an additional analytical verification problem. The plate strip had dimensions

$$L = 100, \qquad b = 20$$

with material properties

$$E = 200000, \qquad \nu = 0.25, \qquad t = 5.0$$

The left edge was clamped, while the right, top, and bottom edges were free. A transverse line load was applied along the right edge:

$$\bar{q} = -10$$

The total applied force was therefore

$$P = \bar{q}b = (-10)(20) = -200$$

This matched the total assembled finite element load.

The finite element result was compared against beam-strip theory. The plate strip was treated approximately as a cantilever beam with second moment of area

$$I = \frac{bt^3}{12}$$

and cross-sectional area

$$A = bt$$

The Euler–Bernoulli tip deflection is

$$w_{\mathrm{EB}} = -\frac{PL^3}{3EI}$$

For a thick beam, the Timoshenko tip deflection includes both bending and shear deformation:

$$w_{\mathrm{Timo}} = -\left(\frac{PL^3}{3EI} + \frac{PL}{\kappa GA}\right)$$

where

$$G = \frac{E}{2(1+\nu)}$$

and

$$\kappa = \frac{5}{6}$$

For the 12×4 mesh seen in Figure 8, the numerical tip displacement at the center of the free edge was

$$w_{\mathrm{tip,num}} = -1.58754$$

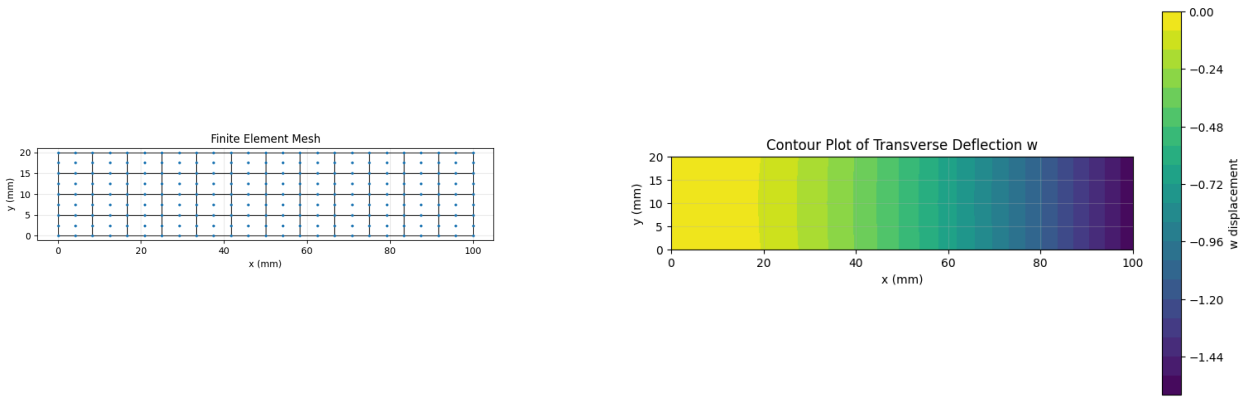

Figure 8: Finite element mesh and contour plot of transverse displacement for the cantilever plate strip.

The Euler–Bernoulli beam solution was

$$w_{\mathrm{EB}} = -1.60000$$

and the Timoshenko beam solution was

$$w_{\mathrm{Timo}} = -1.60300$$

The percent error relative to the Timoshenko solution was

$$0.9643\%$$

Table 7: Cantilever plate strip convergence results.

n_x	n_y	Elements	DOFs	$w_{\mathrm{tip,num}}$	Error vs. Timoshenko (%)
4	2	8	127	-1.582655	1.2692
8	4	32	427	-1.586975	0.9997
12	4	48	627	-1.587542	0.9643
16	6	96	1191	-1.587837	0.9459
20	8	160	1931	-1.587955	0.9385

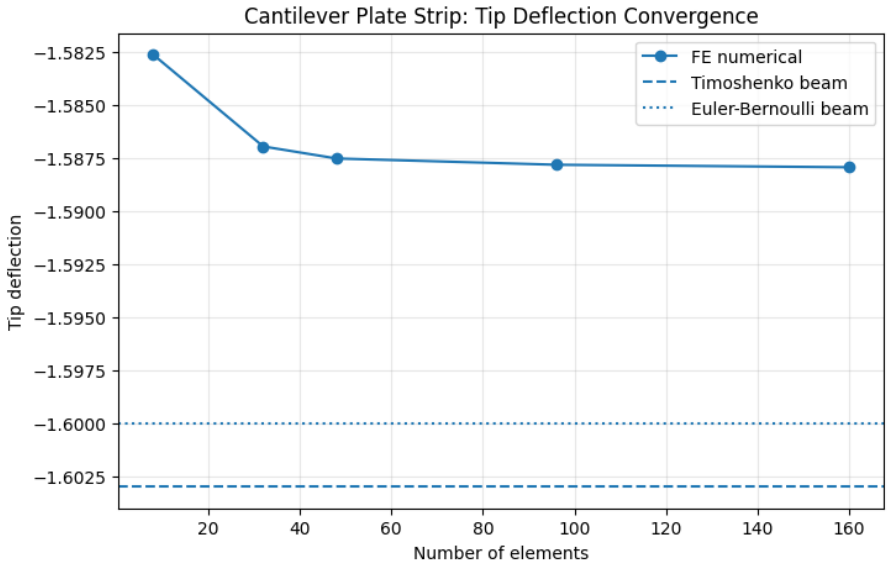

Figure 9: Convergence of tip deflection for the cantilever plate strip under an end line load. The finite element solution is compared against Euler–Bernoulli and Timoshenko beam-strip reference solutions.

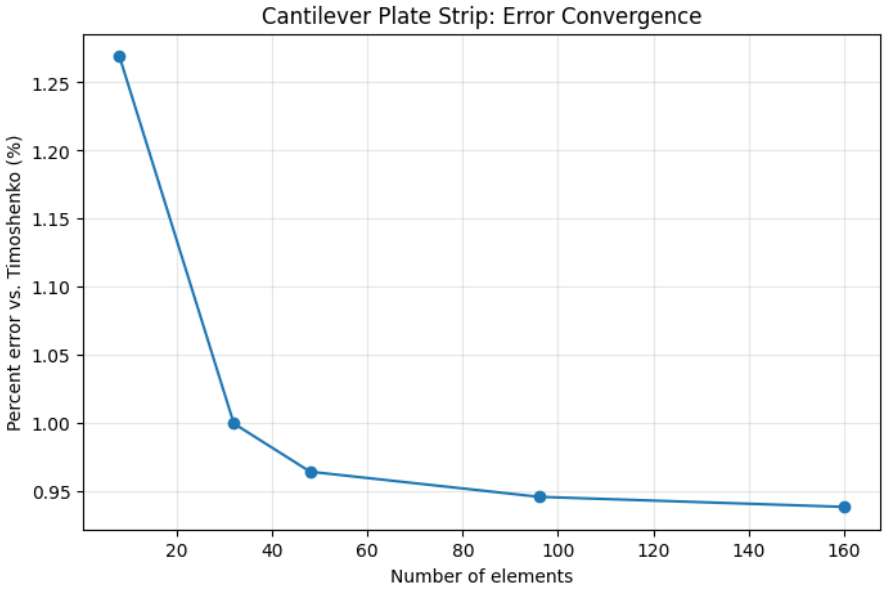

Figure 10: Percent error convergence for the cantilever plate strip under end line load. The finite element solution remains within approximately 1% of the Timoshenko beam-strip solution.

This example was used as an engineering verification case rather than an exact plate benchmark. It checks the implementation of clamped boundary conditions, free boundaries, edge line loading, bending deformation, and transverse shear deformation. The solution is expected to be close to, but not identical to, the Timoshenko beam result.

F Appendix: Analytical Example Problem – Clamped Circular Plate

A clamped circular plate with a concentrated load at the center was used as an analytical benchmark problem. The problem was modeled using one quadrant of the circular plate with symmetry conditions along the two straight radial edges and clamped boundary conditions along the circular edge.

For a clamped circular plate with center load P, the analytical thin-plate center deflection is

$$w_0 = -\frac{PR^2}{16\pi D}$$

where

$$D = \frac{Et^3}{12(1-\nu^2)}$$

The numerical result was normalized as

$$\frac{16\pi D|w_0|}{PR^2}$$

so that the exact thin-plate target value is 1.0.

For the 8×8 mesh, the numerical center displacement was

$$w_{0,\text{num}} = -0.027844$$

compared to the analytical thin-plate value

$$w_{0,\text{exact}} = -0.027976$$

This gives a normalized numerical value of 0.99525.

Table 8: Circular plate benchmark convergence results.

n_x	n_y	Elements	DOFs	Normalized Center Deflection
2	2	4	71	0.906132
4	4	16	227	0.984215
8	8	64	803	0.995250
12	12	144	1731	0.996497
16	16	256	3011	0.997017

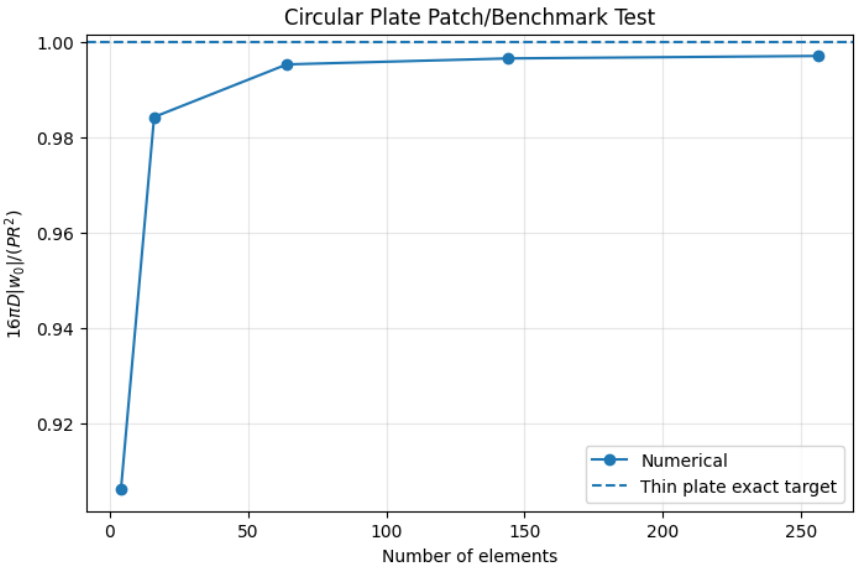

Figure 11: Circular plate benchmark convergence. The normalized center deflection approaches the analytical thin-plate target value of 1.0 as the mesh is refined.

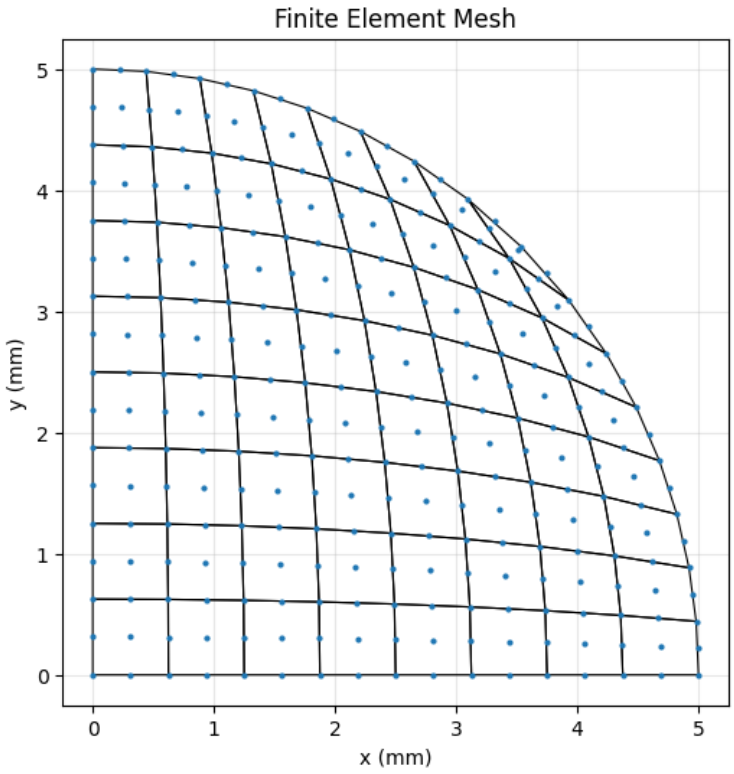

Figure 12: Quarter circular plate mesh used for the 8×8 benchmark model.

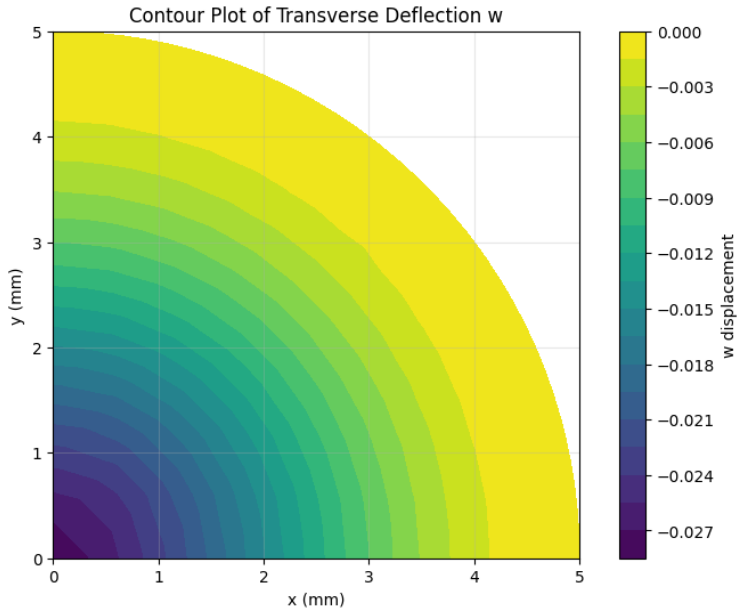

Figure 13: Transverse displacement contour for the clamped circular plate benchmark. The maximum downward displacement occurs at the center, as expected for a centrally loaded clamped circular plate.

G Appendix: Load and Boundary Condition Details

The applied project load was modeled as a consistent line load along the upper edge of the rectangular cutout:

$$\boldsymbol{f}_e = \int_{\Gamma_e} \boldsymbol{N}^T \bar{q}\, ds$$

Only the transverse displacement DOFs received force contributions. The total applied load was checked as

$$\bar{q}L = (-1000\ \text{N/mm})(250\ \text{mm}) = -250000\ \text{N}$$

The clamped boundaries were enforced by setting

$$w = 0, \qquad \theta_1 = 0, \qquad \theta_2 = 0$$

on the top and left edges. The right and bottom edges were left free.

H Appendix: Code Availability and Solver Function Catalogue

The full Python implementation is provided in the accompanying GitHub repository:

`https://github.com/kateykim/CE_222_final_project`

The repository includes the full Jupyter notebook implementation, verification examples, mesh convergence studies, plotting functions, and figures used in this report. Table 9 summarizes the main solver functions, including their signatures, purposes, and verification methods.

Table 9: Summary of solver functions, purpose, and verification method.

Function signature	**Brief description**	**Verification**
`generate_nodes(nx, ny, Lx, Ly)` → `nodes, Nx, Ny`	Creates the structured Q9 background grid with coordinates and active status for each node.	Mesh unit tests
`get_q9_connectivity(i, j, Nx)` → `connectivity`	Returns the 9-node element connectivity in the ordering $[BL, BR, TR, TL, MB, MR, MT, ML, C]$.	Connectivity unit tests
`point_inside_rect_cutout(x, y, cutout, tol)` → `Boolean`	Checks whether a point lies strictly inside a rectangular cutout.	Cutout unit tests
`deactivate_nodes_in_cutouts(nodes, cutouts)` → `nodes`	Deactivates nodes strictly inside cutouts while keeping cutout boundary nodes active.	Cutout unit tests
`centroid_inside_any_cutout(centroid, cutouts)` → `Boolean`	Checks whether an element centroid falls inside any cutout, used to remove cutout elements.	Mesh and cutout tests
`on_rect_cutout_boundary(x, y, cutout, tol)` → `Boolean`	Checks whether a point lies on the rectangular cutout boundary.	Boundary tests
`tag_boundaries(nodes, Lx, Ly, cutouts, tol)` → `boundary`	Tags nodes on exterior boundaries and cutout boundaries.	Boundary tests
`generate_mesh(nx, ny, Lx, Ly, cutouts)` → `nodes, elements, boundary, Nx, Ny`	Main mesh generation routine for structured Q9 meshes with optional rectangular cutouts.	Mesh tests

Function signature	Brief description	Verification
`is_q9_center_node(node_id, Nx)` → `Boolean`	Identifies Q9 center nodes, which do not receive active w-DOFs in the Q8/Q9 element.	DOF tests
`assign_dofs(nodes, Nx)` → `dof_map, ndof`	Assigns global DOF numbers for w, θ_1, θ_2, with no w-DOF at Q9 center nodes.	DOF tests
`get_element_dofs(conn, dof_map)` → `edofs`	Builds the local-to-global element DOF list for the 26-DOF heterosis element.	DOF and stiffness tests
`shape_Q8(xi, eta)` → `N`	Returns Q8 serendipity shape functions for transverse displacement w.	Shape function tests and patch tests
`shape_Q8_derivatives(xi, eta)` → `dN_dxi, dN_deta`	Returns parent-coordinate derivatives of Q8 shape functions.	Derivative tests and patch tests
`shape_Q9(xi, eta)` → `N`	Returns Q9 shape functions for geometry and rotations.	Shape function tests and patch tests
`shape_Q9_derivatives(xi, eta)` → `dN_dxi, dN_deta`	Returns parent-coordinate derivatives of Q9 shape functions.	Derivative tests and patch tests
`jacobian(coords, dN_dxi, dN_deta)` → `J`	Forms the Jacobian matrix for the isoparametric mapping.	Jacobian tests and patch tests
`jacobian_inverse(J, tol)` → `detJ, invJ`	Computes the determinant and inverse of the Jacobian and checks for singular mappings.	Jacobian tests
`map_derivatives(dN_dxi, dN_deta, invJ)` → `dN_dx, dN_dy`	Maps shape function derivatives from parent coordinates to physical coordinates.	Jacobian tests and patch tests
`jacobians(coords, dN_dxi, dN_deta)` → `J, detJ, dN_dx, dN_dy`	Convenience routine that computes the Jacobian, determinant, and physical derivatives.	Jacobian tests and patch tests
`map_Q8_derivatives_with_Q9_geometry (coords, xi, eta)` → `J, detJ, dN_dx_w, dN_dy_w`	Maps Q8 displacement derivatives using the Q9 geometry mapping.	Patch tests
`gauss_1D(n)` → `pts, wts`	Returns 1D Gauss quadrature points and weights.	Quadrature tests
`gauss_2D(n)` → `points, weights`	Builds the tensor-product 2D Gauss quadrature rule.	Quadrature tests
`build_B_bending(dN_dx, dN_dy, conn, dof_map)` → `Bb`	Constructs the bending strain-displacement matrix for curvatures.	B-matrix tests and patch tests
`build_B_shear(dN_dx_w, dN_dy_w, N_theta, conn, dof_map)` → `Bs`	Constructs the transverse shear strain-displacement matrix.	B-matrix tests and patch tests
`build_D_matrix(E, nu, t, kappa)` → `Db, Ds`	Builds bending and shear constitutive matrices for Reissner–Mindlin theory.	Material matrix tests
`element_stiffness(coords, conn, dof_map, E, nu, t)` → `Ke, edofs`	Computes the element stiffness matrix by integrating bending and shear stiffness terms.	Stiffness tests and examples
`assemble(K, Ke, edofs)` → `K`	Assembles an element stiffness matrix into the global stiffness matrix.	Assembly tests
`assemble_force(F, Fe, edofs)` → `F`	Assembles an element force vector into the global force vector.	Load tests
`edge_shape_Q3(s)` → `N, dN_ds`	Returns quadratic edge shape functions for line-load integration.	Line load tests
`edge_load_vector(edge_coords, qbar)` → `Fe`	Computes a consistent edge line-load vector.	Load tests and examples
`element_edge_nodes(conn, side)` → `edge_nodes`	Returns the 3-node quadratic edge from Q9 connectivity.	Load tests
`edge_edofs(edge_nodes, dof_map)` → `edofs`	Returns active transverse displacement DOFs for an edge load.	Load tests
`is_edge_on_rect_cutout_top(edge_coords, cutout, tol)` → `Boolean`	Identifies whether an element edge lies on the upper cutout boundary.	Load location tests
`apply_cutout_top_line_load(F, elements, nodes, dof_map, cutout, qbar)` → `F`	Applies the project line load along the top edge of the cutout.	Total load test
`fixed_dofs(boundary, dof_map, clamped_sides)` → `fixed`	Returns all DOFs to constrain for clamped boundaries.	Boundary condition tests
`bc(K, F, fixed)` → `K, F`	Applies essential boundary conditions by modifying the global stiffness matrix and force vector.	Boundary condition tests and examples
`assemble_system(nodes, elements, dof_map, ndof, E, nu, t)` → `K, F`	Builds the global stiffness matrix and initializes the global force vector.	Assembly tests and examples